

Frontend Integration Requirements

Asset Management System

Project Overview

The Asset Management System is an internal organizational tool designed to track hardware inventory (laptops, monitors, etc.), manage categories, and assign assets to employees. The backend is built on Node.js/Express with MS SQL Server.

General Guidelines for Frontend Integration

- **Base API URL:** `http://localhost:5000/api` (Update this when deploying).
- **Authentication:** The system uses JWT for security. Upon successful login, store the token in `localStorage` or `sessionStorage`.
- **Authorization Header:** For all protected routes (Assets, Categories, Allocations), pass the token as `Authorization: Bearer <your_token>`.

2. Required UI Screens & Flow

Login Screen

- **Fields:** Username, Password.
- **Crucial Note:** Do NOT build a Signup or Register page. Account creation is strictly restricted to IT Admins via the backend.

Dashboard / Asset Inventory

- A data table displaying all assets: Asset Tag, Name, Category Name, Vendor, Purchase Date, Price, Warranty, Status.
- A button to **Add New Asset**.

Add Asset Form

- Requires a dropdown for **Category** (fetch from GET Categories API).
- Fields: Asset Tag, Asset Name, Vendor Name, Purchase Date (Datepicker), Price (Numeric), Warranty Expiry (Datepicker).

Category Management (Settings)

- A simple screen to view existing categories and a modal/input to add a new category.

Employee Directory (Read-Only)

- A list displaying employees.
- No **Add Employee** button should exist, as this data is synced from an external HR server.

3. API Contract & Endpoints

A. Authentication Module

Endpoint	Method	Payload (Request Body)	Success Response	Notes
/auth/login	POST	{ "username": "...", "password": "..."} }	{ "success": true, "token": "...", "user": {...} }	Saves JWT token for future requests.

B. Employee Module

Endpoint	Method	Payload	Success Response	Notes
/employees	GET	None	{ "success": true, "count": 10, "data": [{id, empid, emp_name, emp_code, emp_dept}] }	Use this to populate Employee dropdowns for Asset Allocation.

C. Category Module

Endpoint	Method	Payload	Success Response	Notes
/categories	GET	None	{ "success": true, "data": [{CategoryID, CategoryName}] }	Use to populate Category dropdowns in the Add Asset form.
/categories	POST	{ "categoryName": "Laptop" }	{ "success": true, "message": "..."} }	Adds a new category to the database.

D. Asset (Inventory) Module

Endpoint	Method	Payload (Request Body)	Success Response	Notes
/assets	GET	None	{ "success": true, "data": [{AssetID, AssetTag, AssetName, CategoryName, Status...}] }	Returns a joined table (includes CategoryName instead of just CategoryID).
/assets	POST	{ "assetTag": "...", "assetName": "...", "categoryID": 1, "vendorName": "...", "purchaseDate": "YYYY-MM-DD", "price": 50000, "warrantyExpiryDate": "YYYY-MM-DD" }	{ "success": true, "message": "Asset added successfully!" }	categoryID is mandatory. Status defaults to 'Available'.

4. Pending Modules (To Be Integrated Next)

- **Asset Allocations (/allocations):** APIs for assigning an asset to an employee and processing returns.
- **Asset Bills (/bills):** APIs for logging purchase invoices against specific assets.

Integration Note: The frontend should treat the API contract above as the source of truth for endpoint paths, HTTP methods, request payloads, and response structure. The base URL can be changed later when the backend is deployed to a shared machine/server.